STATS 143: Research in Statistics

Architecture Design & Hyperparameter Optimization of Convolutional Neural Networks for CIFAR-10 Image Classification

Group Members

Allison Lynn
Cynthia Du
Bryan Mui
Ryan So
Mian Xie

Instructor: Hongquan Xu

Date: April 2, 2026

Abstract

This study investigates the design and optimization of convolutional neural networks (CNNs) for CIFAR-10 image classification using Neural Architecture Search (NAS) and Differentiable Architecture Search (DARTS). To better isolate the effects of model structure and training dynamics, architecture selection and hyperparameter tuning are treated as separate stages. Bayesian Optimization is applied to efficiently tune key training parameters, including learning rate, batch size, weight decay, optimizer choice, and momentum.

The results show that while NAS provides a computationally efficient baseline, DARTS consistently identifies higher-performing architectures, achieving validation accuracies near 85%. The final DARTS-based model achieves a test accuracy of 81.1%, demonstrating strong generalization to unseen data despite the expected gap between validation and test performance. However, these gains come at a significantly higher computational cost, with DARTS requiring substantially longer training times per trial compared to NAS. Overall, this work highlights the trade-off between model performance and computational efficiency and demonstrates that combining architecture search with Bayesian Optimization leads to stable and effective CNN configurations within practical resource constraints.

1 Introduction

This project focuses on image classification using the CIFAR-10 dataset, a widely used benchmark in computer vision and machine learning research. The dataset consists of 60,000 color images of size 32×32 pixels distributed across 10 object categories, with 50,000 training images and 10,000 test images. Its standardized structure and balanced class distribution make it well-suited for evaluating and comparing different classification algorithms in the field of machine learning.

The primary objective of this study is to design and optimize a convolutional neural network (CNN) that achieves high classification accuracy on the CIFAR-10 dataset. We investigate how architectural choices and training parameters influence model performance, including the number of convolutional layers, kernel sizes, pooling strategies, optimizer selection, and the number of training epochs. In particular, we explore the effects of hyperparameter tuning on predictive accuracy and model stability.

Through systematic experimentation, this project aims to identify an effective network architecture and training strategy. By analyzing how these parameters affect learning behavior and generalization performance, this study provides insight into why certain design choices lead to improved classification outcomes. In addition, this project aims to compare and contrast the different search methods and provide recommendations based on ease of implementation, model performance, and training time. Our findings suggest that DARTS combined with Bayesian Optimization provides the best balance between performance and stability, albeit at a higher computational cost.

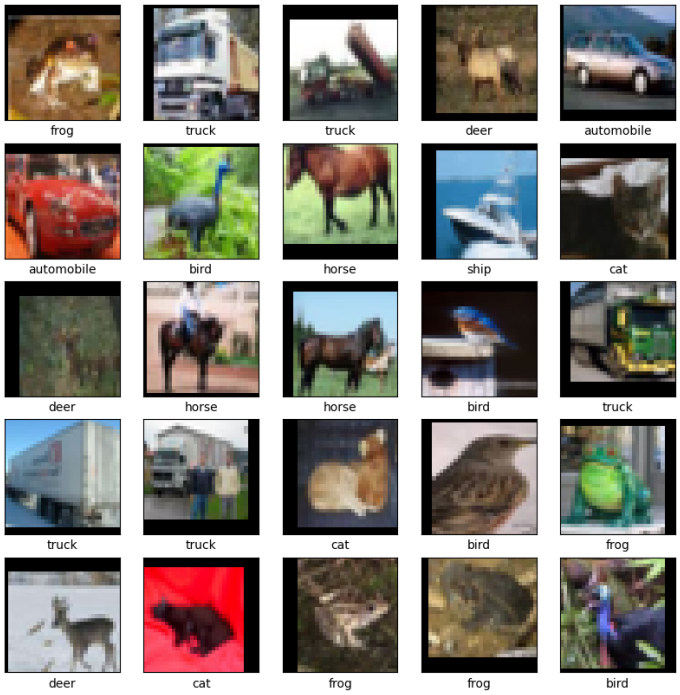

Figure 1: Sample images from Cifar-10 Dataset

2 Methodology

This project investigates automated neural architecture search (NAS) methods for designing high-performance convolutional neural networks on CIFAR-10. Architecture search and hyperparameter optimization are treated as two distinct stages: architecture determines model capacity, while hyperparameters govern training dynamics. Separation of these stages enables controlled optimization, clearer attribution of performance gains, and improved computational efficiency. We first identify strong architectures and then refine the training process through hyperparameter tuning.

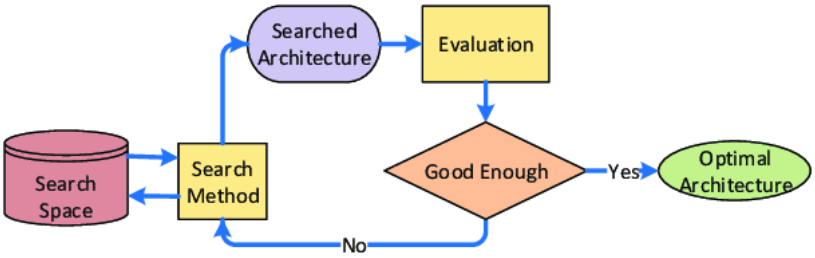

Figure 2: Methodology for attaining architecture

2.1 Neural Architecture Search (NAS)

NAS is adopted as the foundational framework to automate CNN design within a predefined search space consisting of convolutional depth, kernel size, pooling type, and activation function. Candidate architectures are sampled and evaluated using validation accuracy. Kyriakides and Margaritis (2020) describe the various search space methods in NAS, which include global, micro/cell, and hierarchical search spaces (Kyriakides & Margaritis, 2020, pp. 2–4). Due to limited computational resources, random search is used for initial exploration, providing broad coverage of the global search space at a substantially lower cost than an exhaustive grid search. Using a global search space along with random search allowed for a balance of model expressiveness and computational feasibility. The random search included architectures with 2–5 convolutional layers, filter sizes in 16, 32, 64, kernel sizes in 3, 5, and either ReLU or Leaky ReLU activations. Both max pooling and average pooling were included to allow flexible spatial downsampling while keeping the overall complexity manageable.

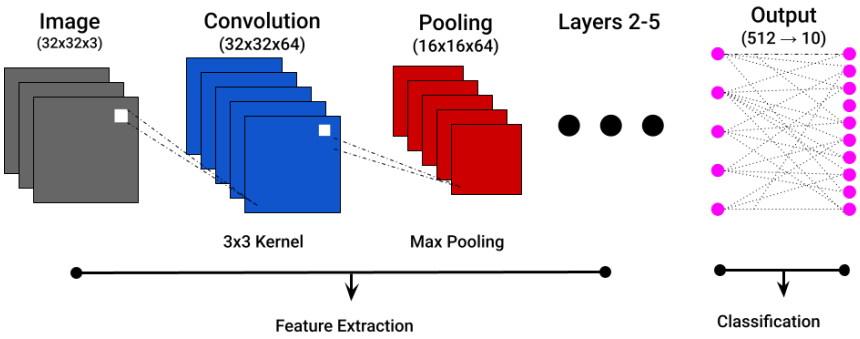

Figure 3: Sample architecture from the defined search space. The model is a three-layer convolutional neural network using LeakyReLU activations and max pooling.

2.2 Differentiable Architecture Search (DARTS)

Differentiable Architecture Search (DARTS) further improves efficiency by relaxing discrete architectural choices into a continuous space, representing candidate operations as weighted mixtures. Architecture parameters and network weights are optimized jointly using bilevel optimization, with weights updated on training loss and the architecture on validation loss. This gradient-based search identifies high-performing architectures without repeatedly retraining separate models.

According to the research introducing DARTS as a meaningful architecture search method (Liu et al., 2019), previous attempts to train a neural network on 100 epochs of CIFAR-10 yielded a competitive model performance of 96%–97% classification accuracy. This is comparable to other architecture search methods like NAS (2,000 GPU-day cost), AmoebaNet-A (3,150 GPU days), and ENAS (0.5–4 GPU days) by being within 1% of other methods' testing accuracy, while only requiring the researchers to train for 1.5 GPU days (Liu et al., 2019, p. 7). During the training of the architectures, a five-layer limit was implemented so that the DARTS algorithm could be compared on even ground to the NAS approach; similarly, it had filter sizes in 16, 32, 64. The different candidate operations had varying kernel sizes, activation functions, and pooling methods that were optimized simultaneously.

2.3 Bayesian Optimization

After architecture selection, Bayesian Optimization is applied to tune key training hyperparameters, specifically learning rate and batch size. This approach utilizes a probabilistic surrogate model, which is an inexpensive approximation compared to fully training the model, to guide the evaluation toward the most promising configurations, significantly improving sample efficiency compared to manual or grid search methods. According to (Snoek et al., 2012), a prior distribution is used to form an assumption about the model performance, and a following aquisition function is used to determine the next point of evaluation. The Gaussian prior is recommended because of it's described "flexibility and tractability," and is a powerful function that is largely used in modern Bayesian Optimization implementations (Snoek et al., 2012, p. 2).

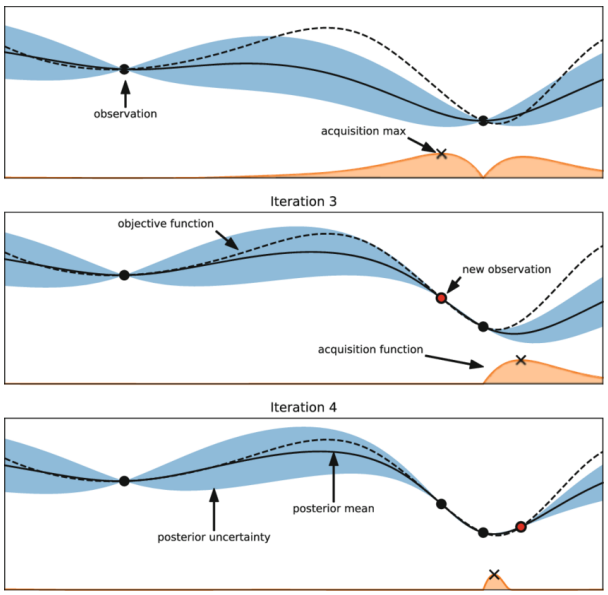

Figure 4: Bayesian Optimization on a 1D Function. The goal is to minimize the dashed line using a Gaussian process surrogate (predictions are the black line, blue line is uncertainty) by maximizing the acquisition function.

2.4 Performance Optimization Plan

Performance optimization is conducted in two stages: architectural search followed by hyperparameter tuning. We first explore CNN architectures using Neural Architecture Search (NAS) within a constrained search space to ensure computational feasibility. The number of convolutional layers is limited to two to five, with kernel sizes of 3×3 and 5×5, max or average pooling, and ReLU or Leaky ReLU activations. These constraints reflect common CNN design practices while preventing excessive spatial downsampling from repeated pooling.

Differentiable Architecture Search (DARTS) is then applied to refine kernel size, pooling type, and activation function within the fixed layer configuration. Architectural parameters are optimized jointly with network weights using gradient-based bilevel optimization, enabling efficient identification of strong architectures without retraining separate models.

Finally, Bayesian Optimization is used to tune training hyperparameters, specifically learning rate and batch size. By modeling validation performance probabilistically and selecting configurations via expected improvement, this stage systematically refines training performance. Final architectures are trained from scratch under standardized protocols and compared using test accuracy, training time, and model complexity.

For the learning rate, we explore a range between 10^{-4} and 3×10^{-2} using a logarithmic scale to account for the high sensitivity of deep neural networks to orders of magnitude in gradient updates. This range is centered around the 0.025 initial learning rate utilized in the DARTS benchmark for CIFAR-10 (Liu et al., 2019). Batch size is treated as a categorical variable (32, 64, or 128) to balance gradient stability with the hardware memory constraints of our single-GPU setup, noting that the original study utilized a batch size of 64 during the search phase and 96 for final evaluation.

The choice of optimizers includes both Momentum SGD and AdamW to determine the most effective convergence strategy for the larger, 20-cell evaluation network. While Momentum SGD with a parameter of 0.9 is the established standard for image classification on CIFAR-10 (Liu et al., 2019), AdamW is included as a candidate for its superior handling of weight decay in complex, high-dimensional search spaces. Momentum is tuned within a window of 0.8 to 0.99 to allow the model to better navigate the loss landscape during the 50-epoch retraining process. By optimizing these parameters after the architecture is fixed, we mitigate the risk of discrepancy between the search and evaluation phases, ensuring the final model achieves a competitive test error rate comparable to the $2.76 \pm 0.09\%$ reported in the original differentiable architecture search findings (Liu et al., 2019).

2.5 Computation and Computing Frameworks

Many search space algorithms for convolutional neural networks require intensive computational resources in order to run effectively. Although the computational requirements aren't very intensive for training a single network, building optimization networks, grid searches, or trying any array of models can easily build up. Consider a search that attempts to find the best combination of 5 parameters each with 5 choices. A grid search would require training and evaluating 3125 separate networks which isn't feasible or efficient.

During the course of this study, the original research objectives were refined to account

for practical computational constraints. Several convolutional neural network architectures reported in the literature require extensive training times, ranging from approximately 1.5 to 4 days of computation time on high-performance GPU hardware, which exceeded the computational resources available for this project.

As a result, certain methods, such as NASNet-A and AmoebaNet-A, could not be empirically evaluated due to their runtime requirements. Consequently, the scope of the research was adjusted to emphasize not only classification accuracy but also computational efficiency, allowing for a meaningful and realistic comparison of CNN architectures that can be trained within accessible computing environments. This revised focus ensures that the conclusions drawn remain both practically relevant and scientifically rigorous.

ACCESS resources, a national research cyberinfrastructure ecosystem aimed to provide institutions with no-cost access to high-performance GPU time. Through the sponsor of ACCESS and Purdue Anvil AI, we were able to acquire 400 GPU-hours for the course of research, which was enough computational overhead for our project.

The Purdue Anvil AI, starting in 2021, originally contained 64 NVIDIA A100 GPUs distributed across 16 nodes. Upon it's expansion in 2025, Anvil AI added 84 Nvidia 80GB H100 SXM GPUs distributed across 21 Dell PowerEdge XE9640 compute nodes.

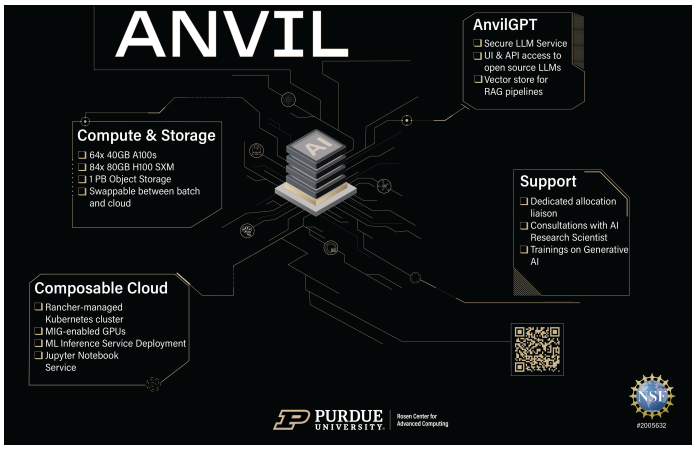

Figure 5: Features and Specifications of Anvil

3 Results

We split the training dataset into 45,000 training samples and 5,000 validation samples to ensure that the test dataset remained unseen during model development. The training and validation sets were used for architecture search and subsequent hyperparameter tuning, with model selection based on validation accuracy. In addition to performance metrics, we recorded the training time for each model to assess the computational cost associated with different architectures. The final selected model was then evaluated on the held-out test dataset to measure generalization performance.

3.1 Architecture Selection

3.1.1 Neural Architecture Search (NAS)

We first determined candidate architectures using Neural Architecture Search (NAS) within a randomly sampled grid search space. Table 1 presents the five top-performing models identified during the NAS search, along with their corresponding architectures, validation accuracies, and training times. The results show that architectures with four to five convolutional layers consistently achieved the highest validation performance, suggesting that moderate network depth is effective for CIFAR-10 classification. Additionally, training times remained relatively similar across the top models, indicating that variations in architecture complexity did not substantially increase computational cost within the explored search space. Figure 6 illustrates the architecture of the highest-performing model selected during the NAS process, highlighting the configuration that achieved the best validation accuracy among all candidate models. Across all NAS experiments, the total runtime for training 50 architecture trials over 50 epochs was 15,356.54 seconds (approximately 4.27 hours), with an average training time of 307.13 seconds (approximately 5.12 minutes) per trial.

Table 1: Top 5 CNN Architectures Identified by NAS Random Search

Layers	Filters	Kernels	Activation	Pooling	Val Acc	Train Time (s)
5	[64,64,64,64,64]	[5,3,5,5,5]	LeakyReLU	Avg	0.8146	309.93
5	[32,64,64,64,64]	[5,3,3,3,3]	ReLu	Max	0.8102	313.28
4	[64,64,32,64]	[3,3,3,5]	ReLu	Max	0.8090	313.00
5	[64,64,64,32,32]	[5,3,5,5,5]	ReLu	Max	0.8058	310.69
4	[64,32,64,64]	[3,3,5,5]	LeakyReLu	Avg	0.8026	312.50

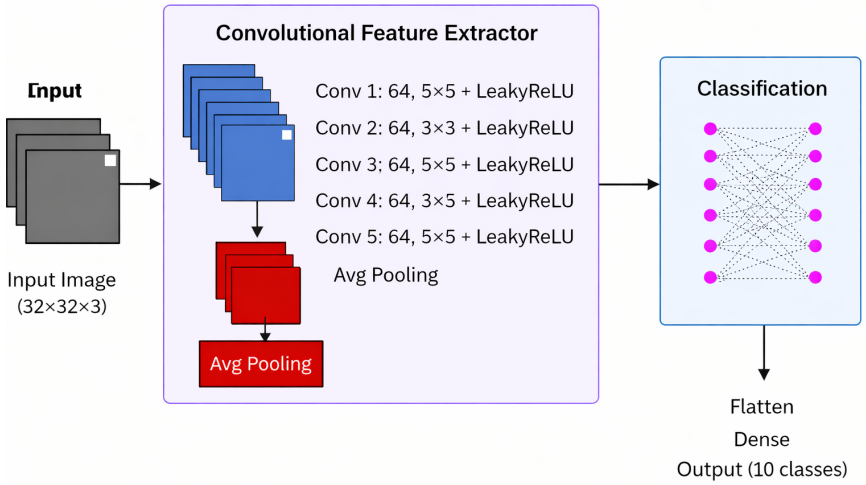

Figure 6: NAS Selected Architecture *(Diagram generated with AI assistance)*

A total of 50 architecture trials were evaluated, with each model trained for 50 epochs. This configuration allowed sufficient training iterations for model convergence while enabling exploration of a diverse set of architectures under the constraints of available GPU resources. The best-performing architecture was selected as the trial achieving the highest validation accuracy. Figure 7 reports the training and validation accuracy trajectories for this selected model across 50 epochs.

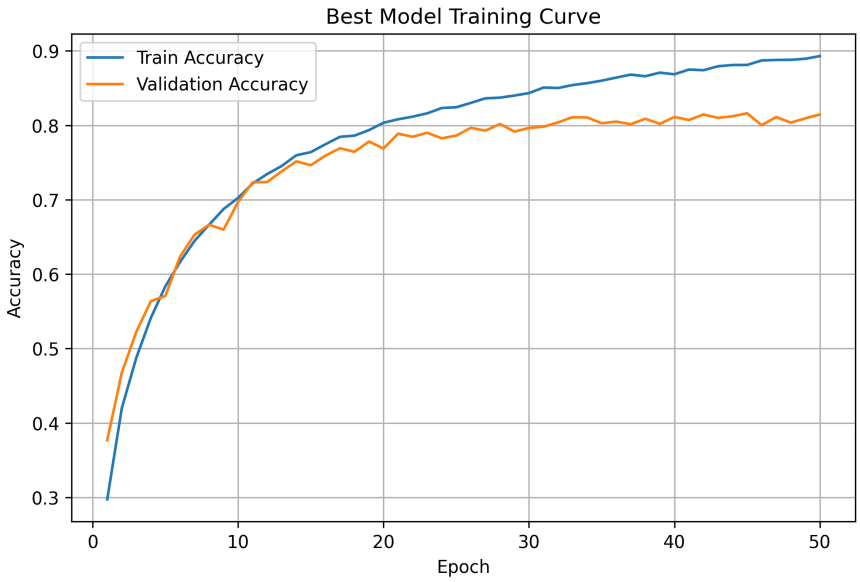

Figure 7: Training Convergence of the Best NAS-Discovered Architecture

As shown in Figure 7, accuracy increases rapidly during the early training stages, indicating that the convolutional layers are quickly learning useful feature representations from the input images. After approximately the midpoint of training, validation accuracy begins to plateau around 0.80–0.82, while training accuracy continues to increase and reaches approximately 0.89 by the final epoch. The growing gap between training and validation accuracy in later epochs suggests mild overfitting, where the model continues to improve its fit to the training images but achieves limited additional gains on unseen validation data. Overall, the results indicate that this CNN architecture converges stably and achieves strong classification performance, with diminishing improvements once validation accuracy stabilizes.

3.1.2 Differentiable Architecture Search (DARTS)

After completing the NAS random search experiments, we applied Differentiable Architecture Search (DARTS), a gradient-based subset of NAS, to evaluate whether it could improve model performance. Because DARTS iteratively learns the operations within each layer while keeping the overall network structure fixed, the number of layers and filter sizes were held constant during the search process. Specifically, we adopted the best-performing configuration identified during the NAS phase, consisting of a five-layer architecture with 64 filters in each layer. Within this framework, the learned operations correspond to the kernel size, activation function, and pooling type selected for each layer. Table 2 presents

the three top-performing models identified by DARTS, including their learned operations for each layer, validation accuracies, and training times measured over 50 epochs across 50 trials.

Table 2: Top CNN Architectures Identified by DARTS

Learned Operations	Val Acc	Train Time (s)
L1: Conv3 + LeakyReLU + MaxPool L2: Conv3 + ReLU + MaxPool L3: Conv5 + ReLU + MaxPool L4: Conv5 + ReLU + MaxPool L5: Conv3 + LeakyReLU + MaxPool	0.8512	829.17
L1: Conv3 + ReLU + MaxPool L2: Conv3 + ReLU + MaxPool L3: Conv5 + LeakyReLU + MaxPool L4: Conv5 + ReLU + MaxPool L5: Conv3 + LeakyReLU + MaxPool	0.8504	822.19
L1: Conv3 + LeakyReLU + MaxPool L2: Conv3 + ReLU + MaxPool L3: Conv5 + ReLU + MaxPool L4: Conv5 + LeakyReLU + MaxPool L5: Conv5 + LeakyReLU + MaxPool	0.8504	827.27

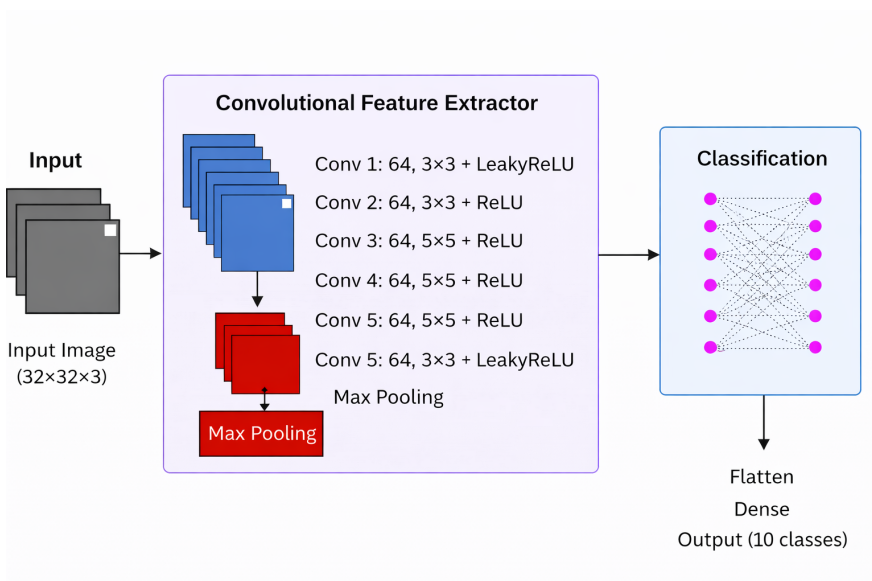

Figure 8: DARTS Selected Architecture *(Diagram generated with AI assistance)*

We observe that training times increased substantially for this experiment. The total runtime for the same number of epochs and trials was 40,844.758 seconds (approximately 11.35 hours), with an average training time of 816.89 seconds (approximately 13.61 minutes)

per trial. Although the longer runtime results in a computational cost almost three times that of the NAS search, the models achieve improved validation accuracy, reaching approximately 85%. Additionally, while both LeakyReLU and ReLU activations appear across different layers of the selected architectures, MaxPool is consistently chosen as the preferred pooling operation.

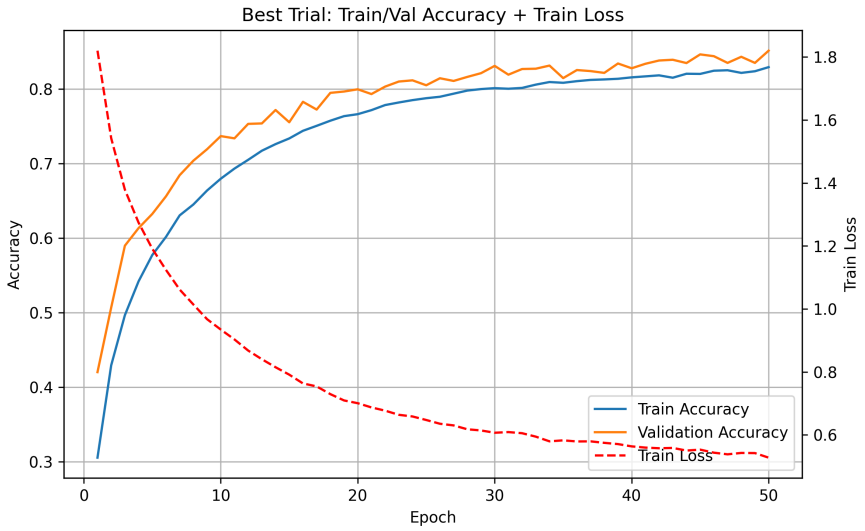

Figure 9: Convergence Behavior of the Best DARTS Architecture: Training and Validation Accuracy with Training Loss

As shown in Figure 9, the best-performing configuration identified through DARTS demonstrates stable training dynamics across epochs. Training and validation accuracy both increase rapidly during the early epochs, indicating that the model quickly learns useful feature representations from the CIFAR-10 images. Interestingly, validation accuracy consistently remains slightly higher than training accuracy throughout most of the training process, suggesting that the model generalizes well to unseen validation data and may benefit from the regularization effects introduced by pooling layers and stochastic training. At the same time, training loss decreases steadily across epochs, reflecting effective optimization of the model parameters. Toward later epochs, both accuracy curves begin to stabilize near approximately 0.84–0.85, indicating that the model approaches convergence. Overall, these results suggest that the DARTS-derived architecture achieves strong classification performance while maintaining stable convergence and good generalization behavior.

3.2 Hyperparameter Tuning

Following architecture selection, we performed hyperparameter optimization on the best-performing architectures identified by both the NAS and DARTS methods. Bayesian Optimization was used to efficiently explore the hyperparameter space, tuning key training parameters including learning rate, weight decay, batch size, optimizer choice, and momentum. For each selected architecture, 25 optimization trials were conducted, with every

configuration trained for 50 epochs. Figure 10 illustrates the training and validation accuracy trajectories across epochs for the best-performing Bayesian Optimization trial applied to the NAS-selected architecture.

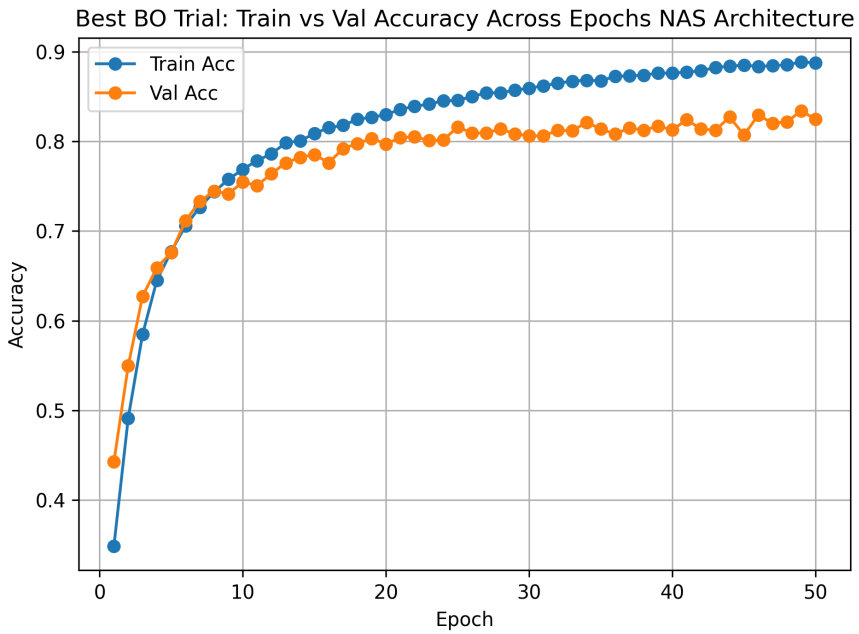

Figure 10: Training and Validation Accuracy Across 50 Epochs for the Best Bayesian Optimization Configuration Applied to the NAS Architecture

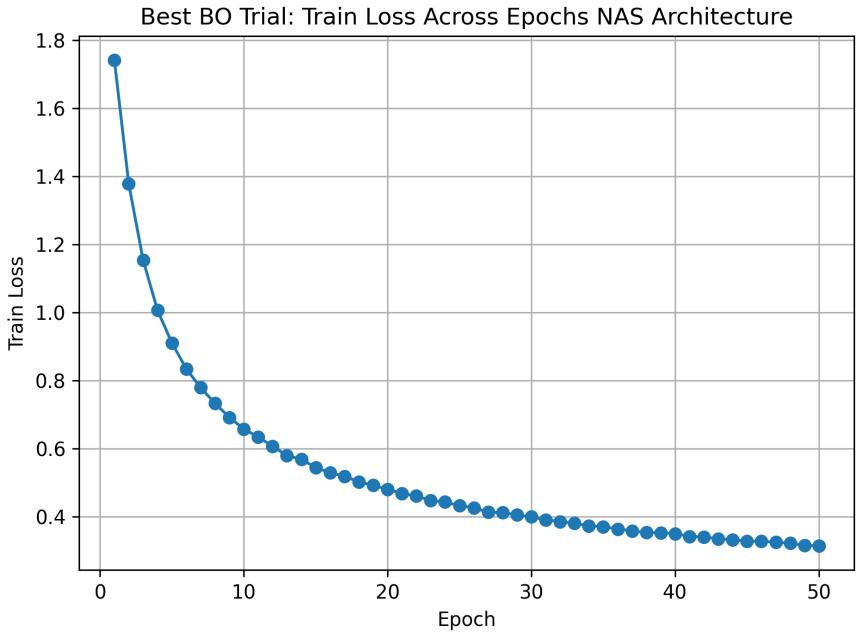

Figure 11: Training Loss Across 50 Epochs for the Best Bayesian Optimization Configuration Applied to the NAS Architecture

As shown in Figures 10 and 11, the model exhibits stable convergence under the optimized hyperparameter configuration. The selected hyperparameters consist of a learning rate of 1.238×10^{-3}, weight decay of 2.2×10^{-5}, batch size of 32, the Adam optimizer, and a momentum parameter of 0.857, which together promote efficient and well-conditioned optimization. The learning rate enables rapid initial updates without instability, while weight decay provides regularization to control parameter growth, and the batch size balances gradient stability with computational efficiency. Adam with momentum further supports smooth and adaptive convergence. This configuration enables a fast initial learning phase followed by gradual refinement, with steadily decreasing training loss. A slight divergence between training and validation trends indicates mild overfitting on the training set, but overall reflects a well-tuned setup that supports stable training dynamics and generalization, with validation accuracy reaching 83.4%. Bayesian Optimization effectively identifies this calibrated hyperparameter set.

Moving to hyperparameter tuning for the architecture identified by the DARTS search, we applied the same Bayesian Optimization procedure to the DARTS-selected configuration. Figure 12 illustrates the training and validation accuracy trajectories across epochs for the best-performing Bayesian Optimization trial for the DARTS architecture.

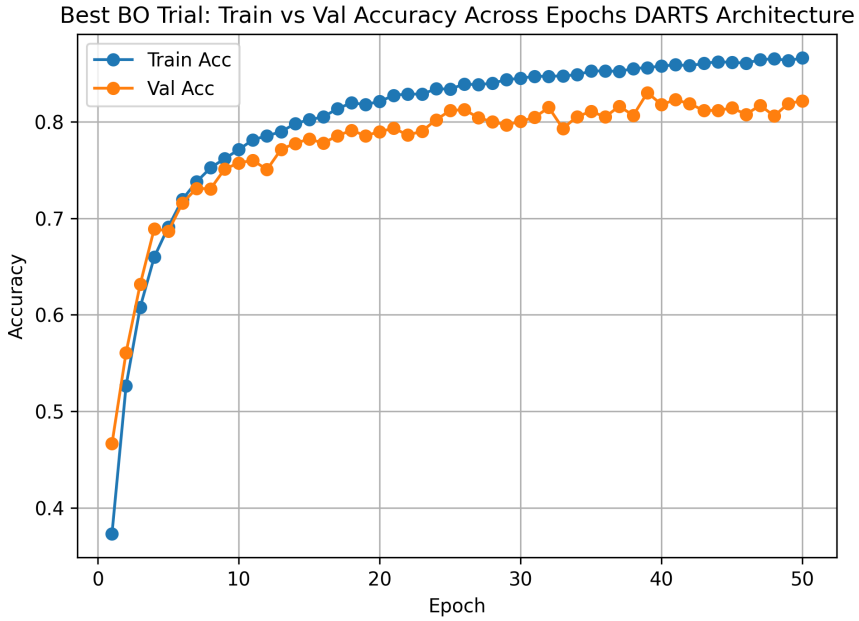

Figure 12: Training and Validation Accuracy Across 50 Epochs for the Best Bayesian Optimization Configuration Applied to the DARTS Architecture

As shown in Figure 12, the DARTS-based model demonstrates similarly stable convergence under its optimized hyperparameter configuration. The selected hyperparameters include a learning rate of 6.73×10^{-4}, weight decay of 4.84×10^{-4}, batch size of 64, the Adam optimizer, and a momentum parameter of 0.809, which together support smooth and efficient optimization. Compared to the NAS configuration, the slightly lower learning rate and higher weight decay introduce stronger regularization, contributing to controlled parameter updates and reduced variance during training. Training dynamics closely mirror

those observed previously, with rapid initial learning followed by gradual refinement and steadily decreasing loss reaching a final validation accuracy of 83%. These results indicate that DARTS, combined with Bayesian Optimization, identifies a stable and effective hyperparameter setting.

3.3 Final Training & Evaluation on Test Data

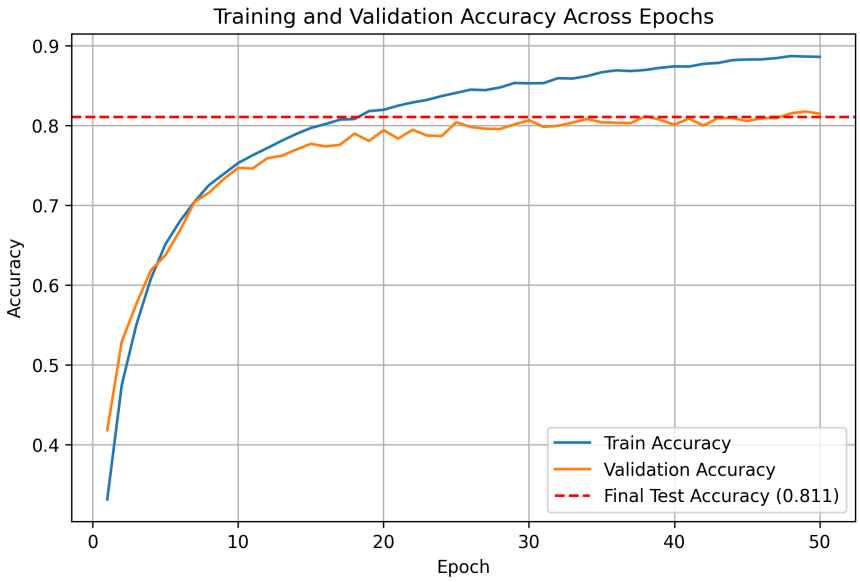

Figure 13: Final Training, Validation, and Test Accuracy on Selected Model

Given its superior validation performance and efficient architecture search capabilities, DARTS was selected as the final model configuration, consistently outperforming standard NAS while incurring higher computational cost. Figure 13 presents the final testing performance of the selected DARTS architecture and corresponding hyperparameters. After model selection, the network was retrained and evaluated on the held-out test set at the final (50th) epoch. The observed test accuracy of 81.1% is slightly lower than the validation performance (approximately 83%–85%), reflecting the expected generalization gap when evaluating on previously unseen data. Importantly, the test set was never exposed during training or model selection, ensuring an unbiased estimate of out-of-sample performance. This result confirms that the selected configuration generalizes well beyond the validation set and maintains stable performance under true deployment conditions, while highlighting the trade-off between improved performance and increased computational expense relative to NAS.

4 Limitations

Despite the systematic design of our architecture search and hyperparameter optimization framework, several practical limitations affect the scope and generalizability of this study.

A primary constraint is limited access to high-performance computing resources, which restricts the ability to train and evaluate a wide range of CNN architectures on advanced GPU hardware. As a result, the architectural search space was confined to models that could be trained within accessible environments, emphasizing computational feasibility alongside predictive performance. To maintain tractability, the search space was limited to variations in convolutional depth (2–5 layers), kernel sizes (3×3 and 5×5), pooling types (max and average), and activation functions (ReLU and Leaky ReLU). While these choices reflect common CNN configurations, excluding more complex components such as residual connections, dense connectivity, or attention mechanisms may prevent the discovery of higher-performing models. Consequently, the selected architecture likely represents a locally optimal solution within a constrained search space rather than a global optimum. Similarly, Bayesian Optimization was conducted over a restricted hyperparameter space with a limited number of trials, which may have further constrained exploration of the parameter landscape.

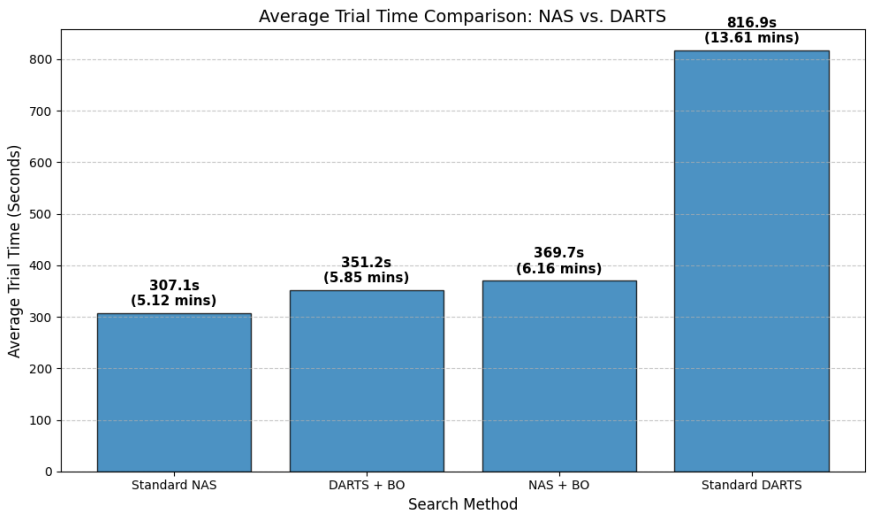

Figure 14: Average Trial Time Comparison between DARTS and NAS

In the end, we evaluated four approaches: Standard NAS and Standard DARTS, each with 50 trials, and NAS with Bayesian Optimization and DARTS with Bayesian Optimization, each with 25 trials, all evaluated over 50 epochs. Figure 14 presents average trial time for each method, highlighting clear differences in computational cost. Standard DARTS is significantly more expensive, with an average trial time of approximately 816.9 seconds (13.61 minutes), more than double that of all other methods. In contrast, Standard NAS is the most efficient at 307.1 seconds per trial (5.12 minutes), while NAS with Bayesian Optimization and DARTS with Bayesian Optimization fall in a similar range at 369.7 and 351.2 seconds, respectively. Bayesian Optimization introduces only a modest increase in computational cost relative to Standard NAS, while substantially reducing cost compared to Standard DARTS. Overall, the results highlight a key trade-off between search method and computational efficiency, with DARTS offering strong performance at a higher per-trial cost.

5 Conclusion

In this study, we investigated the impact of neural architecture design and hyperparameter optimization on convolutional neural network performance for CIFAR-10 image classification. By separating architecture search and hyperparameter tuning into two stages, we were able to systematically evaluate how structural and training decisions influence model behavior, convergence, and generalization.

Our results show that while Neural Architecture Search (NAS) provides a computationally efficient baseline, Differentiable Architecture Search (DARTS) consistently identifies higher-performing architectures, achieving validation accuracies near 85%. When combined with Bayesian Optimization, both approaches benefit from improved training stability and well-calibrated hyperparameter configurations, leading to reliable and consistent performance. The final DARTS-based model achieved a test accuracy of 81.1%, demonstrating strong generalization to unseen data despite the expected gap between validation and test performance.

However, these improvements come with a notable computational trade-off. DARTS incurs significantly higher per-trial training time compared to NAS, making it more resource-intensive despite its performance gains. In contrast, Bayesian Optimization introduces only modest computational overhead while improving parameter selection, making it a practical enhancement to both search methods.

Overall, this study highlights the importance of balancing model performance with computational efficiency in neural architecture design. For resource-constrained settings, NAS combined with Bayesian Optimization offers an effective and efficient solution, while DARTS is better suited for performance-critical applications where higher computational cost is justified. Future work could expand the search space to include more advanced architectural components, such as residual connections or attention mechanisms, and explore larger-scale training regimes to further improve model performance.

AI Usage Statement

This work utilized AI-assisted tools to support the writing and editing process, including rewording and refining technical explanations for clarity and coherence. AI tools were also used to assist in the creation of architecture diagrams and to help review and revise code for correctness and efficiency. All final decisions, implementations, and interpretations were completed and verified by the authors.

References

[1] Amit.H. (2024, November 9). *A practical guide to neural architecture search (NAS).* Medium.

[2] Bartz, E., Bartz-Beielstein, T., Zaefferer, M., & Mersmann, O. (2023). *Hyperparameter tuning for machine and deep learning with R.* Springer Nature Singapore.

[3] Kaggle. (2026). *CIFAR-10: Object recognition in images.*

[4] Krizhevsky, A. (2009). *CIFAR-10 and CIFAR-100 datasets.* University of Toronto.

[5] Kyriakides, G., & Margaritis, K. (2020). *An introduction to neural architecture search for convolutional networks.*

[6] Liu, H. (2025). *Differentiable architecture search for convolutional and recurrent networks.* GitHub.

[7] Liu, H., Simonyan, K., & Yang, Y. (2019). DARTS: Differentiable architecture search. *International Conference on Learning Representations (ICLR).*

[8] Pham, H., Guan, M., Zoph, B., Le, Q., & Dean, J. (2018). Efficient neural architecture search via parameter sharing. *International Conference on Machine Learning (ICML).*

[9] Salehin, I., Islam, M. S., Saha, P., Noman, S. M., Tuni, A., Hasan, M. M., & Baten, M. A. (2023). AutoML: A systematic review on automated machine learning with neural architecture search. *Journal of Information and Intelligence, 2*(1).

[10] Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems (NeurIPS).*